جامعة بجاية
Tasdawit n Bgayet
Université de Béjaïa

UNIVERSITÉ A/MIRA DE BÉJAÏA

DÉPARTEMENT D'INFORMATIQUE

3ème ANNÉE INGÉNIEUR – MODULE COMPILATION

Année Universitaire 2025-2026

COMPTE RENDU DE PROJET

Mini-Compilateur Python : Analyse Lexicale et Syntaxique

GUELAACHE HOCINE

ALLOUCHE MOKRANE

BOUKHENAK KAMEL

BOUNOUA AXIL

Date de soumission : 15 Avril 2026

SOMMAIRE

Table des matières

SOMMAIRE **2**

1. INTRODUCTION ET OBJECTIFS **3**

Environnement technique 3

2. SPÉCIFICATIONS DU LANGAGE MINI-PYTHON **3**

2.1 Déclarations et Affectations 3

2.2 Opérateurs 3

2.3 Structures de Contrôle 4

2.4 Spécificité Python : Gestion de l'Indentation 4

3. ARCHITECTURE DU SYSTÈME **4**

4. ANALYSEUR LEXICAL (LEXER) **4**

4.1 Fonctionnement Général 4

4.2 Algorithme de Gestion de l'Indentation 5

4.3 Types de Tokens Reconnus 5

5. ANALYSEUR SYNTAXIQUE (PARSER) **5**

5.1 Méthode d'Analyse 5

5.2 Grammaire Formelle 5

5.3 Gestion des Erreurs 6

6. INTERFACE GRAPHIQUE **6**

6.1 Composants Visuels 6

6.2 Coloration Sémantique 7

6.3 Identité Visuelle 7

7. TESTS ET VALIDATION **7**

7.1 Test 1 : Code Python Valide.....7
7.2 Test 2 : Détection d'Erreur Syntaxique.....9
8. CONCLUSION ET PERSPECTIVES.....10
8.1 Bilan.....10
8.2 Limites Actuelles.....10
8.3 Perspectives d'Évolution.....10

1. INTRODUCTION ET OBJECTIFS

Ce projet s'inscrit dans le cadre du module **Compilation** (3ème année Ingénieur, Département d'Informatique, Université de Béjaïa). L'objectif est d'implémenter un mini-compilateur en Java comprenant :

0. Un **analyseur lexical** (tokenizer) reconnaissant les unités lexicales d'un sous-ensemble de Python
1. Un **analyseur syntaxique** par descente récursive validant la grammaire du langage
2. Une **interface graphique** permettant de visualiser les résultats de l'analyse
3. Une **gestion des erreurs** lexicales et syntaxiques avec localisation précise

Environnement technique

4. Langage :**Java 23** (JDK 23.0.1)
5. Interface :**Swing** (javax.swing)
6. IDE : VS Code

2. SPÉCIFICATIONS DU LANGAGE MINI-PYTHON

Conformément aux exigences du projet, notre langage supporte :

2.1 Déclarations et Affectations

7. **Variables dynamiques**: x = 10, nom = "Python"
8. **Types littéraux**: entiers, flottants, chaînes de caractères, booléens (True/False), None

2.2 Opérateurs

9. **Arithmétiques**: +, -, *, /
10. **Comparaison**: ==, !=, <, >, <=, >=

11. **Logiques**: and, or, not
12. **Affectation**: =

2.3 Structures de Contrôle

13. **Conditionnelles**: if, elif(if else), else
14. **Boucles**: while, for ... in range()
15. **Fonctions**: Définition avec def, retour avec return, appel de fonctions
16. **Entrées/Sorties**: print()

2.4 Spécificité Python : Gestion de l'Indentation

Les blocs de code sont délimités par l'indentation (espaces ou tabulations), gérés par les tokens virtuels **INDENT** (augmentation) et **DEDENT** (diminution).

3. ARCHITECTURE DU SYSTÈME

L'application suit une architecture en trois couches :

Couche	Composants	Description
Présentation	CompilerGUI.java	Interface Swing, éditeur de code, zone de résultats colorisée
Analyse	Lexer.java, Parser.java	Tokenisation et validation grammaticale
Modèle	Token.java, TokenType.java	Structure des données et énumération des tokens
Main	PythonCompiler.java	Point d'entrée et configuration Look & Feel

4. ANALYSEUR LEXICAL (LEXER)

4.1 Fonctionnement Général

Le **Lexer** transforme le code source en une liste de tokens. Il gère :

17. La segmentation en lignes physiques
18. Le calcul des niveaux d'indentation (pile indent Stack)
19. La reconnaissance des mots-clés, identifiants, nombres, chaînes
20. Les commentaires (depuis # jusqu'à la fin de ligne)

4.2 Algorithme de Gestion de l'Indentation

L'algorithme utilise une pile pour suivre les niveaux d'indentation :

21. Si indentation > sommet_pile → INDENT (nouveau bloc)
22. Si indentation < sommet_pile → DEDENT (fermeture de bloc) répété jusqu'au niveau correspondant
23. Vérification de cohérence : un niveau d'indentation doit exister dans la pile

4.3 Types de Tokens Reconnus

24. **Structurels**: NEWLINE, INDENT, DEDENT, EOF
25. **Mots-clés**: if, elif, else, while, for, in, def, return, print, and, or, not
26. **Littéraux**: NUMBER, STRING_LITERAL, TRUE, FALSE, NONE
27. **Opérateurs**: ASSIGN, PLUS, MINUS, TIMES, DIVIDE, COMP_EQ, COMP_NEQ, etc.
28. **Délimiteurs**: LPAREN, RPAREN, COLON, COMMA

5. ANALYSEUR SYNTAXIQUE (PARSER)

5.1 Méthode d'Analyse

Analyseur **récursif descendant** (top-down) implémentant une grammaire LL(1).

5.2 Grammaire Formelle

```
programme     -> instruction* EOF
```

```
instruction    -> affectation | if_stmt | while_stmt | for_stmt |
def_stmt | print_stmt | return_stmt

if_stmt        -> IF expression COLON NEWLINE bloc (ELIF expression
COLON NEWLINE bloc)* (ELSE COLON NEWLINE bloc)?

while_stmt    -> WHILE expression COLON NEWLINE bloc

for_stmt      -> FOR IDENTIFIER IN expression COLON NEWLINE bloc

def_stmt       -> DEF IDENTIFIER LPAREN params? RPAREN COLON NEWLINE
bloc

bloc          -> INDENT instruction+ DEDENT

expression    -> or_expr

or_expr       -> and_expr (OR and_expr)*

and_expr      -> not_expr (AND not_expr)*

not_expr      -> NOT not_expr | comparaison

comparaison   -> arithmetique (op_comp arithmetique)?

arithmetique  -> terme ((PLUS|MINUS) terme)*

terme         -> facteur ((TIMES|DIVIDE) facteur)*

facteur       -> MINUS facteur | atome | LPAREN expression RPAREN

atome           -> NUMBER | STRING_LITERAL | TRUE | FALSE | NONE |
IDENTIFIER (LPAREN args? RPAREN)?
```

5.3 Gestion des Erreurs

Le parser fournit des messages explicites avec numéro de ligne :

29. “Deux-points ':' attendu à la fin de la déclaration”
30. “Indentation attendue après ':'”
31. “Fin de ligne attendue après l'instruction”

6. INTERFACE GRAPHIQUE

L'interface développée avec **Java Swing** présente :

6.1 Composants Visuels

32. **Éditeur de code**: JTextArea avec thème sombre (fond #1e1e28, texte #dcdcdc), police Monospaced

33. **Console de résultats**: JTextPane avec StyledDocument pour la coloration syntaxique des tokens
34. **Barre d'outils**: Boutons "Analyser" (Ctrl+Entrée), "Effacer", "Exemple"
35. **Barre de statut**: Indicateur visuel (Prêt / Syntaxe correcte / Erreur)

6.2 Coloration Sémantique

36. **Vert (#50c850)**: Succès syntaxique
37. **Rouge (#dc5050)**: Erreurs lexicales/syntaxiques
38. **Jaune (#ffd43b)**: Mots-clés, titres de sections
39. **Violet**: Nombres
40. **Orange**: Chaînes de caractères

6.3 Identité Visuelle

Palette aux couleurs officielles de Python : Bleu #376eab et Jaune #ffd43b.

7. TESTS ET VALIDATION

7.1 Test 1 : Code Python Valide

Objectif :Vérifier l'analyse complète d'un programme incluant fonctions, boucles, conditions et opérateurs logiques.

Code source testé :

```
# Définition de fonction
def calculer(a, b):
    total = a + b
    return total

# Appel de fonction
valeur = calculer(10, 20)
print(valeur)
```

```
# Comparaisons et opérateurs logiques
if x >= 5 and y <= 5:
    print('Different')
```

Résultat obtenu :

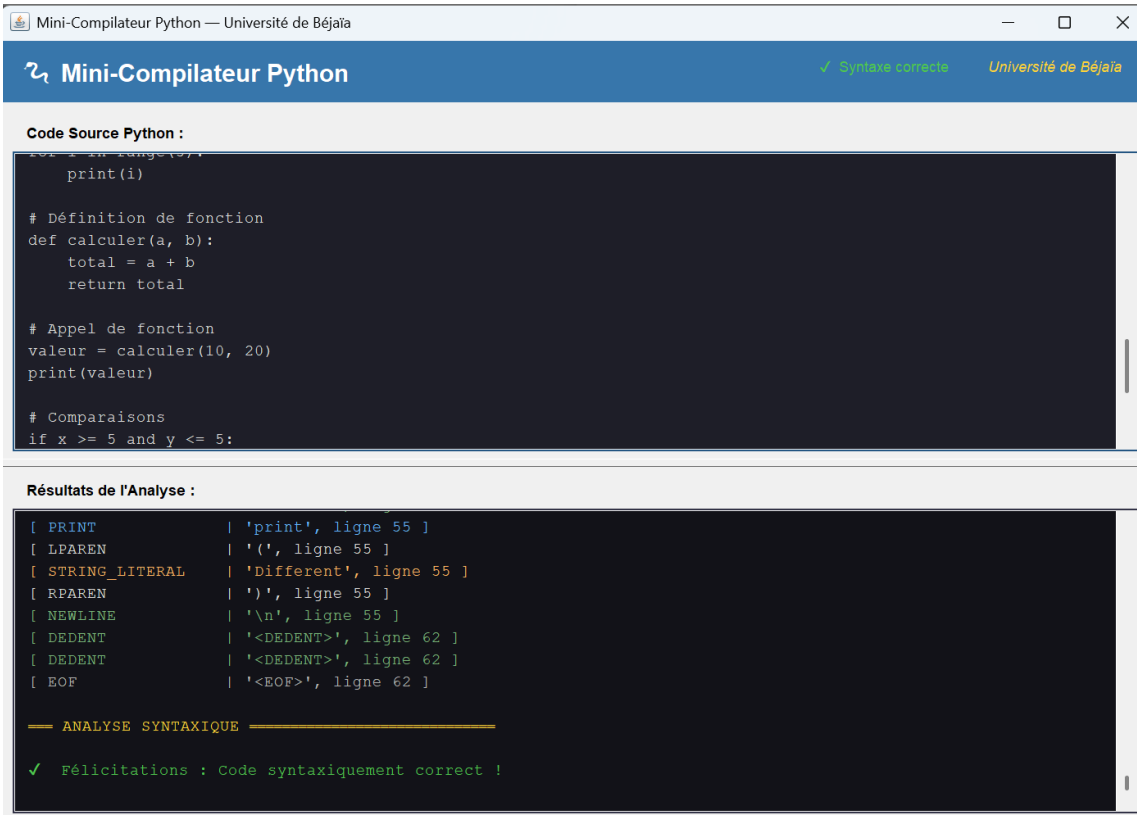


Figure 1 : Résultat de l'analyse d'un code Python valide

Analyse : Le compilateur a correctement tokenisé l'ensemble du programme, reconnu la structure de la fonction calculer, l'appel de fonction, et la condition complexe avec opérateur logique and. La gestion de l'indentation a correctement généré les tokens INDENT et DEDENT pour structurer les blocs.

7.2 Test 2 : Détection d'Erreur Syntaxique

Objectif : Tester la robustesse du parser face à une erreur de syntaxe (opérateur manquant).

Code source erroné :

```
def calculer(a, b):
    resultat = a  b  # Erreur : op\u00e9rateur manquant entre a et b
    return resultat
```

Résultat obtenu :

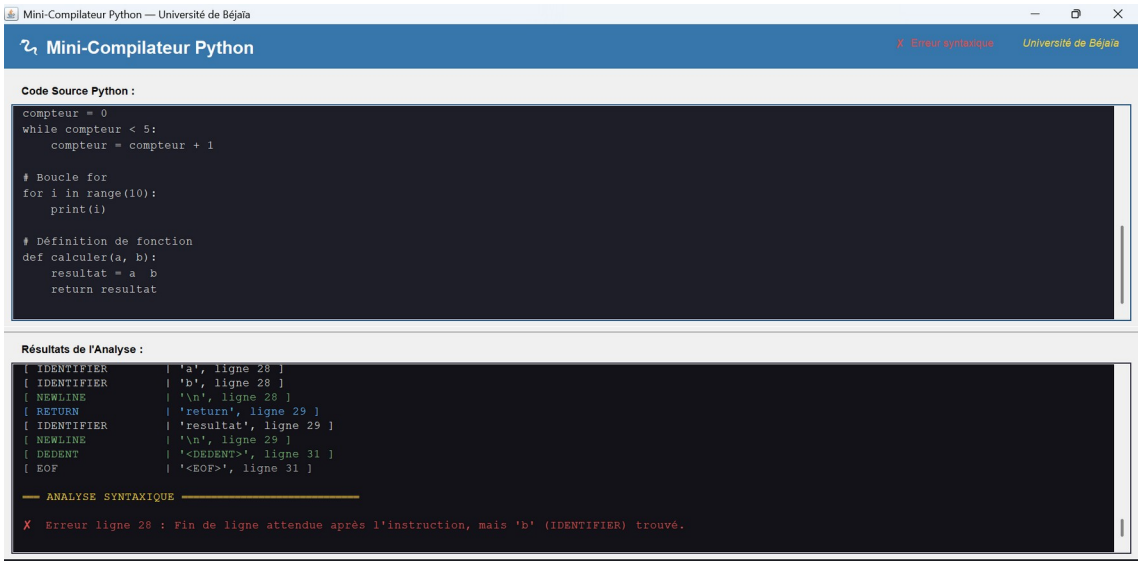


Figure 2 : Détection d'une erreur syntaxique

Analyse : Le lexer ayant réussi à identifier les tokens, l'erreur est purement syntaxique. Le parser attendait une fin d'instruction (NEWLINE) après le premier identifiant a, mais a rencontré un second identifiant b, révélant l'absence d'opérateur binaire. Le message d'erreur est explicite et inclut le type du token rencontré.

8. CONCLUSION ET PERSPECTIVES

8.1 Bilan

Ce mini-compilateur démontre la maîtrise des concepts fondamentaux de la compilation :

41. **Analyse lexicale** efficace avec gestion spécifique de l'indentation Python (off-side rule)
42. **Analyse syntaxique** robuste par descente récursive avec grammaire LL(1) complète
43. **Interface utilisateur** intuitive facilitant l'apprentissage et le débogage

Les tests réalisés valident la capacité du système à traiter des programmes complets (fonctions, boucles imbriquées, conditions complexes) tout en détectant précisément les erreurs avec des messages informatifs incluant les numéros de ligne.

8.2 Limites Actuelles

44. Analyse statique uniquement (pas de génération de code ou d'exécution)
45. Pas de vérification sémantique (types des variables, variables non déclarées)
46. Sous-ensemble limité de Python (pas de classes, d'héritage, de gestion d'exceptions)

8.3 Perspectives d'Évolution

47. **Analyse sémantique**: Implémentation d'une table des symboles pour vérifier la portée des variables et la compatibilité des types
48. **Génération de code**: Production de byte code JVM ou transformation vers un langage intermédiaire
49. **Enrichissement du langage**: Support des structures de données (listes, dictionnaires), des modules import, et des exceptions try/except

Ce projet constitue une base solide pour comprendre les mécanismes internes des compilateurs modernes et pourrait être étendu vers un interpréteur Python complet ou un transpileur vers Java.

Nous vous remercions.